

Guión de exposición — 12 min, 2 personas

Antes de arrancar (setup 5 min antes)

LÁMINA 1 · Portada (0:00 – 0:30) — **Persona A (Hernán)**

LÁMINA 2 · El problema (0:30 – 1:30) — **Persona A**

LÁMINA 3 · Arquitectura en 3 capas (1:30 – 2:45) — **Persona A**

LÁMINA 4 · Matriz de herramientas sin redundancia (2:45 – 4:00) — **Persona A**

LÁMINA 5 · GitHub Actions — CI (4:00 – 5:00) — **Persona B**

LÁMINA 6 · Jenkins — release con gates (5:00 – 6:15) — **Persona B**

LÁMINA 7 · GitOps con Argo CD (6:15 – 7:30) — **Persona B**

LÁMINA 8 · Kyverno — última línea de defensa (7:30 – 8:45) — **Persona B**

LÁMINA 9 · DevSecOps en números (8:45 – 9:45) — **Persona A**

LÁMINA 10 · Aprendizajes y cierre (9:45 – 10:30) — **Persona A**

Resumen de tiempos y responsables

Handoffs (para practicar)

Reglas de oro para la expo

Preguntas probables del profesor + respuestas listas

Guión de exposición — 12 min, 2 personas

Contexto: CI/CD con enfoque DevSecOps — caso Forseti · ISWZ3205 UDLA

Duración total: 12 min hablados + margen para preguntas **Presentadores:**

Persona A (Hernán) + Persona B (compañero) **Distribución:** A ≈ 6 min / B ≈ 6

min **Presentación:** docs/presentacion.pdf (10 láminas)

Antes de arrancar (setup 5 min antes)

Ambos deben tener abierto en su laptop:

1. **Presentación** en pantalla completa (F5 en PowerPoint).

2. **4 pestañas del browser** listas:

- <https://localhost:8080> → Argo CD (ya logueado admin)
- <http://localhost:8090> → Jenkins (ya logueado admin)
- Archivo docs/evidencias/trivy-backend.html
- Archivo docs/evidencias/zap-baseline-report.html

3. **Terminal Git Bash** en `C:\TARIUS-DESARROLLO\productos\forseti-devsecops` con estos comandos ya escritos (no ejecutados) para usar en el momento:

```
kubectl get nodes
kubectl get cpol
kubectl -n forseti-staging get pods
kubectl -n forseti-prod run test-latest --image=nginx:latest --dry-run=server
```

LÁMINA 1 · Portada (0:00 – 0:30) — Persona A (Hernán)

🕒 30 segundos

Qué mostrar: slide portada con el título y ambos nombres.

Qué decir (A):

Buenos días profesora. Somos [nombre A] y [nombre B]. Vamos a presentar nuestra implementación de un pipeline CI/CD con enfoque DevSecOps aplicado a un caso real: **Forseti**, un SaaS de facturación electrónica que emite comprobantes al Servicio de Rentas Internas del Ecuador. Es un producto de TARIUS S.A.S. que ya está en producción y sirve como caso de estudio porque combina tres retos DevSecOps al mismo tiempo: **compliance regulatoria, manejo de certificados digitales y cadencia alta de releases.**

Handoff: ninguno todavía. Cambio a lámina 2.

LÁMINA 2 · El problema (0:30 – 1:30) — Persona A

🕒 60 segundos

Qué mostrar: slide con los 3 bullets de riesgos.

Qué decir (A):

Forseti procesa **datos fiscales sensibles**: RUCs, direcciones, correos electrónicos — todo cubierto por la Ley Orgánica de Protección de Datos Personales del Ecuador. Además, firma los comprobantes con **XAdES-BES** usando certificados digitales `.p12` privados. Un fallo de firma corrompe la validez fiscal del documento y expone a la empresa a sanciones del SRI.

A esto se suma que la base tributaria del SRI cambia dos veces al año, por lo que necesitamos **releases seguros sin frenar la producción**. La pregunta guía del proyecto fue: **¿cómo entregamos rápido sin aflojar la seguridad?** La respuesta que damos es un pipeline con enfoque DevSecOps, que integra controles de seguridad en cada etapa — no al final.

Handoff: sigue A a lámina 3.

LÁMINA 3 · Arquitectura en 3 capas (1:30 – 2:45) — Persona A

🕒 75 segundos

Qué mostrar: diagrama ASCII de las 3 capas.

Qué decir (A):

El pipeline tiene tres capas con responsabilidades **separadas**. Esto es deliberado — no queremos que una sola herramienta haga todo.

La **primera capa** es GitHub Actions. Corre en cada Pull Request y da feedback al desarrollador en menos de cinco minutos. Aquí viven los análisis rápidos: build, tests unitarios, SAST con CodeQL, análisis de dependencias con OWASP Dependency-Check y escaneo de secretos con Gitleaks.

La **segunda capa** es Jenkins. Se dispara cuando el código se mergea a main. Es más pesada — build de imágenes Docker, escaneo Trivy, generación de SBOM con Syft, firma con Cosign, y DAST contra staging con OWASP ZAP. Al final, hace un commit al repositorio GitOps con los nuevos tags de imagen.

La **tercera capa** es Kubernetes gestionado con Argo CD. Argo observa el repo GitOps y reconcilia el cluster automáticamente. Kyverno actúa como admission controller — si un pod no cumple las políticas de seguridad, la API de Kubernetes lo rechaza.

La regla de oro es: cada herramienta hace lo que hace mejor, **nada se pisa**.

Handoff: sigue A a lámina 4.

LÁMINA 4 · Matriz de herramientas sin redundancia (2:45 – 4:00) — Persona A

🕒 75 segundos

Qué mostrar: tabla con las 11 herramientas y su rol único.

Qué decir (A):

Usamos **once herramientas** en total. Todas están justificadas por un rol único.

Para cumplir el requisito **SAST** de la consigna usamos **CodeQL**, que analiza estáticamente el código Java y TypeScript sin ejecutarlo. Para **DAST** usamos **OWASP ZAP**, que ataca la aplicación en runtime buscando vulnerabilidades de

configuración HTTP. Estos dos son el mínimo que pide la consigna — nosotros sumamos cinco categorías más.

Trivy escanea las imágenes Docker construidas buscando CVEs de sistema operativo y librerías empacadas. **Dependency-Check** hace lo mismo pero contra las dependencias declaradas en `pom.xml` y `package-lock.json`. Uno cubre lo que se compila, el otro lo que se declara.

Cosign firma las imágenes con Sigstore. **Kyverno** verifica esa firma en el cluster antes de admitir cualquier pod. Sin ese par, la firma sería decorativa.

Argo CD implementa GitOps: el cluster refleja exactamente lo que dice Git. Y **Jenkins** orquesta las etapas pesadas de release que no queremos en cada PR.

Descartamos explícitamente GitLab CI porque duplicaría GitHub Actions, y Ansible o Puppet porque con Helm ya no aportan.

Le paso la palabra a mi compañero para que muestre el pipeline en acción.

Handoff: A → B. Frase de transición: “Le paso la palabra a mi compañero para mostrar el pipeline en acción.”

LÁMINA 5 · GitHub Actions — CI (4:00 – 5:00) — Persona B

🕒 60 segundos

Qué mostrar: slide con el snippet YAML. Después alterna con el **PR #1 en GitHub** mostrando los checks verdes.

Qué decir (B):

Gracias [nombre A]. La primera capa son los workflows de GitHub Actions. Tenemos cinco archivos YAML en `.github/workflows/` — uno por responsabilidad.

[cambiar al browser: mostrar el PR #1 con los checks verdes]

Este es un Pull Request real que hicimos como demo. Vean que se ejecutan cinco checks en paralelo: Backend build con Java 21 y Maven, Frontend build con Node 22 y Vue 3, CodeQL Analysis para Java y otro para TypeScript, Gitleaks para el escaneo de secretos, y Dependency-Check para las vulnerabilidades de dependencias.

El feedback total al desarrollador es de **menos de cinco minutos**. Si algo se rompe, el PR queda bloqueado — la rama `main` tiene branch protection activado que **exige** que todos estos checks estén verdes antes de permitir el merge.

Handoff: sigue B a lámina 6.

LÁMINA 6 · Jenkins — release con gates (5:00 – 6:15) — Persona B

🕒 75 segundos

Qué mostrar: slide con snippet Groovy del Jenkinsfile. Después alterna con **Jenkins UI en localhost:8090**.

Qué decir (B):

La segunda capa es Jenkins. Es un contenedor Docker que construimos con nuestra imagen personalizada, con Trivy, Syft, Cosign, kubectl y Helm preinstalados.

[cambiar al browser: <http://localhost:8090> mostrar el pipeline]

El Jenkinsfile tiene nueve stages en modo declarativo. Los tres más importantes:

Primero, el stage *Trivy Gate CRITICAL*. Corre `trivy image --severity CRITICAL --exit-code 1 --ignore-unfixed`. Si hay al menos una vulnerabilidad crítica *con parche disponible*, el build se rompe. Los CVEs sin fix se ignoran para no bloquear por cosas fuera de nuestro control.

[cambiar al archivo: <docs/evidencias/trivy-backend.html>]

Este es un reporte real que corrimos contra la imagen del backend. Trivy encontró **siete CRITICAL y ocho HIGH** — todos en librerías Spring y Thymeleaf. Por ejemplo, CVE-2026-40477 es una Server-Side Template Injection en Thymeleaf con fix disponible en 3.1.4. En un release real, este build no pasaría hasta actualizar la dependencia.

Segundo, el stage *Cosign* — *firma + attest SBOM*. Firmamos la imagen con una llave ECDSA propia y adjuntamos el SBOM firmado también. **Tercero**, el stage de OWASP ZAP corre baseline scan contra staging. Y al final, promovemos el tag al repo GitOps.

Handoff: sigue B a lámina 7.

LÁMINA 7 · GitOps con Argo CD (6:15 – 7:30) — Persona B

🕒 75 segundos

Qué mostrar: slide con la config de Argo. Después **Argo CD UI en <https://localhost:8080>**.

Qué decir (B):

La tercera capa es Argo CD implementando GitOps.

[cambiar al browser: https://localhost:8080]

Esta es la UI de Argo. La regla que aplicamos es: el cluster **no acepta cambios directos** — todo lo que corre acá viene de un commit al repositorio GitOps separado, que se llama `forseti-devsecops-gitops`.

Argo observa ese repo cada tres minutos, detecta el commit, hace `helm template` con los `values.yaml` del ambiente correspondiente y aplica los manifests. Vean que la aplicación `forseti-staging` está en estado **Synced** y **Healthy** — corresponde exactamente al último commit del repo GitOps.

[hacer click en la app y mostrar el tree]

Este tree muestra todos los recursos Kubernetes gestionados por Argo: deployments de backend y frontend, statefulset de Postgres, services, ingress, secrets, PVC. Los tres podcitos están corriendo.

El beneficio concreto es que la **auditoría es automática**: cada estado del cluster corresponde a un commit en Git. Los rollbacks se hacen con `git revert`. Y separamos el repo de código del repo GitOps porque no queremos que el pipeline que hace bump de tags pueda tocar el código de negocio.

Handoff: sigue B a lámina 8.

LÁMINA 8 · Kyverno — última línea de defensa (7:30 – 8:45) — Persona B

🕒 75 segundos

Qué mostrar: slide con la tabla de 5 políticas. Después **terminal Git Bash**.

Qué decir (B):

Aunque el pipeline firme las imágenes, no confiamos ciegamente en el pipeline. Kyverno es un admission controller que valida cada pod **en el momento de creación**.

Tenemos **cinco políticas enforce**: verifica firma Cosign, exige requests y limits, prohíbe el tag `:latest` en producción, prohíbe containers privilegiados, y exige `runAsNonRoot`.

[cambiar a la terminal]

Voy a intentar crear un pod con tag `:latest` en el namespace de producción — algo que debería estar bloqueado.

```
kubectl -n forseti-prod run test-latest --image=nginx:latest --dry-run=server
```

[ejecutar el comando]

Vean que Kyverno lo rechaza con tres violaciones al mismo tiempo: la política de `disallow-latest-tag`, la de `require-resource-limits` y la de `require-run-as-non-root`. La API de Kubernetes ni siquiera crea el pod. Este bloqueo se aplica también si un desarrollador intenta hacer `kubectl apply` a mano por fuera del pipeline. Es la **última línea de defensa** del cluster.

Handoff: B → A. Frase de transición: “*Le paso la palabra a mi compañero para cerrar con los resultados.*”

LÁMINA 9 · DevSecOps en números (8:45 – 9:45) — Persona A

🕒 60 segundos

Qué mostrar: tabla de métricas.

Qué decir (A):

Gracias [nombre B]. Estos son los números reales del pipeline ejecutado end-to-end.

El tiempo desde el commit del desarrollador hasta el pod running en staging es de **aproximadamente 22 minutos**. El feedback en el PR llega en menos de **cinco minutos** — el desarrollador ve el error mientras el contexto está fresco.

Por cada release corremos **siete herramientas de seguridad**: CodeQL, Dependency-Check, Gitleaks, Trivy, ZAP, Cosign verify y las cinco políticas de Kyverno.

Después de todos los controles: **cero CVEs críticos fixables no mitigados**, **cero secretos en el historial de Git**, **cero pods sin firma Cosign corriendo en el cluster**, y **cero alertas abiertas en CodeQL**.

Esto no es teoría — es evidencia guardada en la carpeta `docs/evidencias/` del repositorio: reportes SARIF, HTMLs de Trivy y ZAP, SBOMs SPDX con casi cuatro mil componentes, y un archivo con los bloqueos de Kyverno tal como los vimos en vivo.

Handoff: sigue A a lámina 10.

LÁMINA 10 · Aprendizajes y cierre (9:45 – 10:30) — Persona A

🕒 45 segundos

Qué mostrar: slide final con aprendizajes + URL del repo público.

Qué decir (A):

Cerramos con tres aprendizajes clave.

Uno: un pipeline sin *gates* no es un pipeline. Correr las herramientas y publicar reportes sin bloquear nada es *security theater*. Trivy con `--exit-code 1`, Kyverno en modo `Enforce` y `cosign verify` que rompe si falla — esos son los que importan.

Dos: Kubernetes solo aporta valor si hay policies. Un cluster vacío es tan inseguro como un `docker run` sin flags. Kyverno cierra el círculo — sin él, la firma Cosign sería decorativa.

Tres: separar CI rápido de CD pesado hace que ambas cosas mejoren. GitHub Actions puede ser instantáneo porque no carga con las etapas pesadas; Jenkins puede ser exhaustivo porque no bloquea el trabajo diario.

Todo el código, los manifests y los reportes están en **github.com/TARIUS-S-A-S/forseti-devsecops** — repositorio público, licencia MIT. Muchas gracias, quedamos abiertos a preguntas.

Handoff: fin. Ambos quedan al frente para preguntas.

Resumen de tiempos y responsables

#	Lámina	Duración	Habla	Acumulado
1	Portada	0:30	A	0:30
2	El problema	1:00	A	1:30
3	Arquitectura	1:15	A	2:45
4	Matriz herramientas	1:15	A	4:00
5	GitHub Actions	1:00	B	5:00
6	Jenkins	1:15	B	6:15
7	Argo CD	1:15	B	7:30
8	Kyverno	1:15	B	8:45
9	Números	1:00	A	9:45
10	Cierre	0:45	A	10:30
—	Margen preguntas	1:30	ambos	12:00

Handoffs (para practicar)

- **Lámina 4 → 5 (A → B):** “Le paso la palabra a mi compañero para mostrar el pipeline en acción.”
- **Lámina 8 → 9 (B → A):** “Le paso la palabra a mi compañero para cerrar con los resultados.”

Reglas de oro para la expo

1. **No leer las slides.** Las palabras del guión son referencia — decir con las tuyas.
2. **Cuando cambies del slide al browser, avisar:** “Voy a mostrarles esto en vivo...” — así el ojo del profesor sigue el cambio.
3. **Si algo se rompe en vivo (no debería), no entrar en pánico.** Decir: “Este flujo lo tenemos documentado en la evidencia guardada — les muestro el reporte que sacamos ayer” y abrir el HTML de docs/evidencias/.
4. **La segunda persona sostiene el mouse cuando la primera está hablando** — dejar libre para gesticular.
5. **Aprender de memoria las dos primeras frases** de tu lámina — el resto sale solo.
6. **Terminar cada lámina con una frase corta y clara** — el profesor toma nota ahí.

Preguntas probables del profesor + respuestas listas

“¿Por qué usan GitHub Actions Y Jenkins? ¿No son redundantes?” > Cubren tiempos y responsabilidades distintos. Actions da feedback al PR en cinco minutos — no puede tardar más. Jenkins hace release engineering que tarda veinte minutos, algo insoportable en cada PR. Además, la firma con llave propia y el DAST con ZAP requieren credenciales que no queremos exponer al workflow de Actions de cualquier PR. En la industria — Netflix, Airbnb, muchos otros — este patrón dual es estándar.

“¿Por qué eligieron Kyverno y no OPA/Gatekeeper?” > Kyverno usa YAML declarativo. OPA/Gatekeeper requiere aprender Rego, un lenguaje aparte. Para un equipo pequeño, Kyverno se aprende en un día; Gatekeeper toma una semana. Ambos son igual de potentes.

“¿Cómo garantizan que la firma Cosign es válida?” > Dos capas: en el pipeline, `cosign verify` corre antes del push a GHCR y rompe si falla. En el cluster, la política de Kyverno `verify-forseti-image-signatures` valida la firma antes de admitir el pod. Un atacante que suba una imagen sin firmar al registry, aunque logre modificar un `values.yaml`, no logra que el pod se cree.

“¿Qué pasa si Sigstore/Rekor se cae?” > Cosign soporta dos modos: keyless con OIDC de GitHub (que sí depende de Sigstore) y key-based con llave local (que no). Nuestro pipeline usa ambos — GitHub Actions firma keyless, Jenkins firma con llave propia. Los pods se admiten si cualquiera de las dos firmas es válida, así que resistimos una caída de Sigstore.

“¿Cómo escalarían esto a producción real?” > Cambiaríamos kind por un cluster gestionado — EKS, GKE o k3s en VPS. El resto del pipeline es exactamente igual. Argo CD, Kyverno, Cosign y las herramientas de seguridad son las mismas que usan Netflix y Kubernetes upstream. La demo local es una miniatura fiel del estado productivo.